

KUKA youBot — детекция, захват и перестановка кубика по цвету

ROS Noetic · Gazebo · RViz · Ubuntu 20.04 · пакет `youbot_perception` . Редакция 3: движение базы через `planar_move` (Gazebo реально едет), USB-вебкамера, точка В в левом верхнем углу по умолчанию, увеличенный объезд препятствий, обводка кубика на камере.

Что нового в этой редакции.

1. **Gazebo теперь едет.** База управляется плагином `planar_move` и публикует истинную одометрию — RViz и Gazebo синхронны.
2. **Робот не промахивается.** Регулятор движения гасит скорость у цели и делает твёрдый стоп.
3. **USB-вебкамера** (FIFINE K420) вместо 3D-камеры, смотрит в сторону, противоположную руке. Кубик обводится на `/cube/debug_image` .
4. **Точка В** — в левом верхнем углу комнаты по умолчанию.
5. **Объезд цилиндров** усилен (больше зазор).

1. Зависимости (один раз)

Если базовая симуляция уже запускалась, нужны только пакеты для камеры, `planar_move` и захвата.

```
sudo apt update
sudo apt install terminator -y

# ROS-контроллеры (рука/схват/состояние суставов)
sudo apt install ros-noetic-ros-control ros-noetic-ros-controllers \
ros-noetic-gazebo-ros-control ros-noetic-joint-state-controller \
ros-noetic-position-controllers ros-noetic-joint-trajectory-controller -y

# движение базы (planar_move) и камера входят в gazebo_plugins/gazebo_ros
sudo apt install ros-noetic-gazebo-ros ros-noetic-gazebo-plugins -y

# планировщик
sudo apt install python3-scipy python3-numpy -y

# зрение
sudo apt install ros-noetic-cv-bridge ros-noetic-vision-opencv \
ros-noetic-image-transport ros-noetic-tf2-geometry-msgs \
ros-noetic-gazebo-msgs python3-opencv -y

# для реального робота — драйвер USB-вебкамеры
sudo apt install ros-noetic-usb-cam -y

# надёжный захват в Gazebo (если нет — см. примечание)
sudo apt install ros-noetic-gazebo-grasp-plugin -y || true
```

Если `gazebo-grasp-plugin` не ставится — соберите `gazebo-pkgs` из исходников или удалите блок `<plugin name="gazebo_grasp_fix">` из `src/youbot_perception/urdf/youbot_task.urdf.xacro` . Тогда захват держится на трении.

2. Сборка

```
cd ~/youbot_ws
source /opt/ros/noetic/setup.bash
catkin_make
source ~/youbot_ws/devel/setup.bash
# если слетели права на узлы:
bash ~/youbot_ws/src/youbot_perception/scripts/fix_permissions.sh
```

3. Запуск (одна команда)

Панель 1 — roscore (по желанию)

```
source /opt/ros/noetic/setup.bash
roscore
```

Панель 2 — вся задача

```
source /opt/ros/noetic/setup.bash
source ~/youbot_ws/devel/setup.bash
roslaunch youbot_perception pick_place_sim.launch
```

Запускайте именно `pick_place_sim.launch` . Старый `youbot_controller youbot_sim.launch` поднимает робота без камеры и без объектов — кубика, цвета и захвата там нет.

Что произойдёт: откроется Gazebo, заспавнится youBot с USB-камерой, поднимется комната с точкой В в левом верхнем углу, появится кубик случайного цвета и три картонки в С. Через ~10 с робот поедет: **В → распознавание цвета → разворот рукой к кубiku → захват → С → постановка на картонку нужного цвета → возврат в А.**

Признак успеха: в логe миссии `Кубик распознан как green` , затем `Позиционируюсь для захвата...` , `Захват кубика (green)...` и в конце `Миссия завершена: кубик green установлен на картонку green.`

4. Проверка по ходу

Одометрия и движение базы

```
rostopic hz /odom           # ~50 Гц
rostopic echo /cmd_vel      # команды миссии базе
```

Теперь, в отличие от прошлой версии, при ненулевом `/cmd_vel` робот реально едет в Gazebo (`planar_move`), и `/odom` отражает истинное положение.

Камера и детекция

```
rostopic echo /cube/color      # red / green / blue
rqt_image_view /cube/debug_image # кубик ОБВЕДЁН + подпись цвета
rostopic hz /usb_cam/image_raw # идёт ли картинка с камеры
rostopic echo /cube/pose       # позиция кубика в odom
```

RViz

```
roslaunch rviz rviz
# если экран виснет: LIBGL_ALWAYS_SOFTWARE=1 roslaunch rviz rviz
```

Fixed Frame = **odom**. Полезные топики: `/room/markers` (комната, A/B/C), `/mission_planned_path` (Path — путь с объездом, по которому реально едет робот), `/usb_cam/image_raw` (Image), RobotModel, `/odom` (Odometry).

Про синхронность. Раньше RViz «ехал», а Gazebo стоял, потому что одометрия была фейковой (интеграл команд). Теперь одометрию даёт сам Gazebo через `planar_move` — линия `/mission_planned_path` совпадает с движением робота.

5. Геометрия камера/рука (важно понимать)

USB-камера стоит на столике и смотрит **в сторону, противоположную манипулятору**: это «перед» робота для зрения. Рука хватает с другой стороны (по +X базы). Поэтому миссия делает так:

- 1. подъезжает к B, развернувшись **камерой** к кубику — распознаёт цвет и позицию;
- 2. разворачивается **рукой** к кубику и встаёт на дистанцию захвата (`position_for_grasp`);
- 3. хватает и везёт в C.

Если в вашей реальной установке камера закреплена иначе — поправьте крепление (joint `usb_cam_joint` в URDF) и TF в `pick_place_real.launch` .

6. Параметры запуска

Аргумент	По умолч.	Назначение
<code>cell_B</code>	<code>0,4</code>	Ячейка точки B (кубик). <code>0,4</code> = левый верхний угол комнаты 5×5. Формат <code>col,row</code> .
<code>cube_color</code>	<i>(случайно)</i>	Зафиксировать цвет кубика: <code>red / green / blue</code>
<code>seed</code>	<code>42</code>	Зерно ГСЧ (расположение препятствий, цвет кубика)
<code>gui</code>	<code>true</code>	Окно Gazebo (<code>false</code> — без GUI)

Параметры миссии (через `<param>` узла `pick_place_mission`): `robot_radius` (0.40), `safety_margin` (0.15) — зазор объезда; `arm_reach` (0.32) — дистанция захвата; `v_max` (0.22), `pos_tol` (0.06).

Примеры:

```
# красный кубик в левом верхнем углу (по умолчанию):
roslaunch youbot_perception pick_place_sim.launch cube_color:=red

# поставить B в центр:
roslaunch youbot_perception pick_place_sim.launch cell_B:=2,2
```

7. Запуск на реальном роботе

```
roslaunch youbot_perception pick_place_real.launch video_device:=/dev/video0
```

Перед этим: откалибруйте камеру и подайте корректный `/usb_cam/camera_info` ; задайте реальные смещения крепления камеры в `static_transform_publisher` (узел `usb_cam_tf` в launch); подгоните HSV-пороги `COLOR_RANGES` в `cube_detector.py` под освещение. Точки B/C и картонки задайте под вашу сцену (или через `room_generator`).

8. Частые проблемы

Симптом	Решение
Робот стоит в Gazebo, но <code>/odom</code> «едет»	Это была старая проблема механума. Убедитесь, что запущен <code>pick_place_sim.launch</code> (Редакция 3) — база едет через <code>planar_move</code> . Проверьте, что в URDF есть плагин <code>libgazebo_ros_planar_move.so</code> .
Робот проскакивает цель	Решено в этой редакции (гашение скорости + твёрдый стоп). Если осталось — уменьшите <code>v_max</code> или <code>pos_tol</code> .
<code>/cube/color</code> молчит, <code>debug_image</code> пуст	Нет картинки: <code>rostopic hz /usb_cam/image_raw</code> . Если тишина — робот без камеры (запускайте <code>pick_place_sim.launch</code>).

Цвет неверный	Подгоните <code>COLOR_RANGES</code> (HSV) в начале <code>cube_detector.py</code> .
<code>/cube/pose</code> пустой, цвет есть	Нет <code>camera_info</code> или TF до <code>odom</code> . Проверьте <code>rostopic echo /usb_cam/camera_info</code> и дерево TF (<code>rqt_tf_tree</code>).
Робот задевает цилиндр	Увеличьте <code>robot_radius</code> / <code>safety_margin</code> у узла миссии.
Кубик выскальзывает из схвата	Не установлен <code>gazebo_grasp_fix</code> — поставьте плагин или уточните позы <code>ARM_GRASP</code> / <code>GRIPPER_CLOSED</code> .
Рука не дотягивается до кубика	Подстройте <code>arm_reach</code> и позы <code>ARM_*</code> в <code>pick_place_mission.py</code> под высоту кубика.

Итог. Панель 1 (`roscore`) → Панель 2 (`roslaunch youbot_perception pick_place_sim.launch`) → дождаться Gazebo и спавна объектов → проверить `/cube/debug_image` и `/odom`. Миссия стартует автоматически; кубик в левом верхнем углу, цвет — случайный.